UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL
INSTITUTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS

Trabalho Prático 02 - METAHEURÍSTICA

Professor: Marcus

Gabriel Santos Ferreira de Pádua - 4705

Florestal - MG
2024

Sumário

Sumário **2**
1. Introdução **3**
2. Organização **4**
data/: 5
Arquivos .py 6
readme.md 6
3. Desenvolvimento **7**
Soluções iniciais: 7
Critérios de parada: 7
Mutações: 7
Restrições: 8
Penalidades: 8
Seleção e Sobrevivência: 9
4. Resultados: **10**
Análise de Boxplots 13
5. Conclusão **14**
6. Referências **15**

1. Introdução

A otimização de problemas complexos é um tema central na área de Ciência da Computação, especialmente quando se trata de funções com múltiplos mínimos locais e alta dificuldade de exploração do espaço de busca. Nesse contexto, as meta-heurísticas se destacam como estratégias eficientes para encontrar soluções aproximadas de boa qualidade em problemas onde métodos exatos são inviáveis ou computacionalmente custosos.

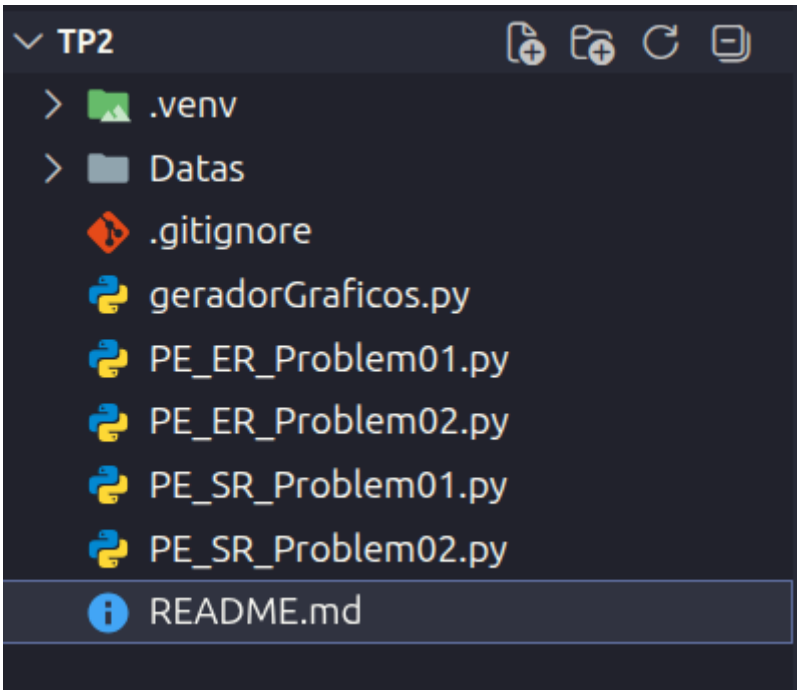
Este trabalho tem como objetivo implementar e analisar o desempenho de algoritmos baseados em metaheurísticas para a minimização de funções objetivas específicas. Em particular, será utilizado o algoritmo de Programação Evolutiva, com diferentes formas de tratamentos de restrições, permitindo uma comparação entre diferentes abordagens de otimização. As implementações serão avaliadas por meio de múltiplas execuções independentes, possibilitando uma análise estatística dos resultados obtidos.

Além disso, serão considerados diferentes intervalos para as variáveis de decisão, o que influencia diretamente na complexidade do problema e no comportamento dos algoritmos. A análise dos resultados será realizada por meio de métricas como valor mínimo, máximo, média e desvio padrão, bem como por representações gráficas, como boxplots.

Dessa forma, este trabalho busca não apenas aplicar conceitos teóricos estudados na disciplina, mas também proporcionar uma compreensão prática sobre o funcionamento e a eficiência das meta-heurísticas em problemas de otimização.

2. Organização

O projeto foi estruturado com uma organização clara de pastas e arquivos, garantindo que todos os elementos necessários para o completo entendimento e utilização estejam acessíveis.

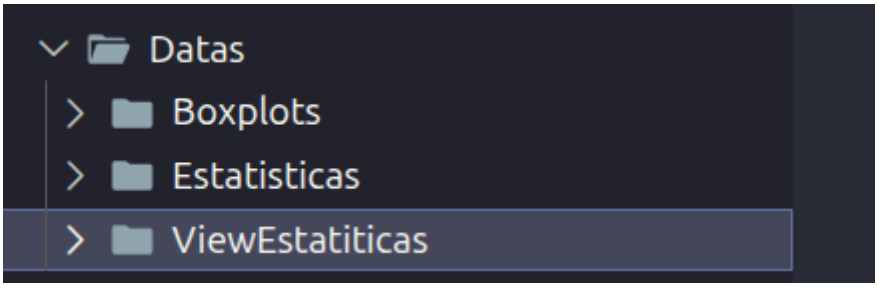


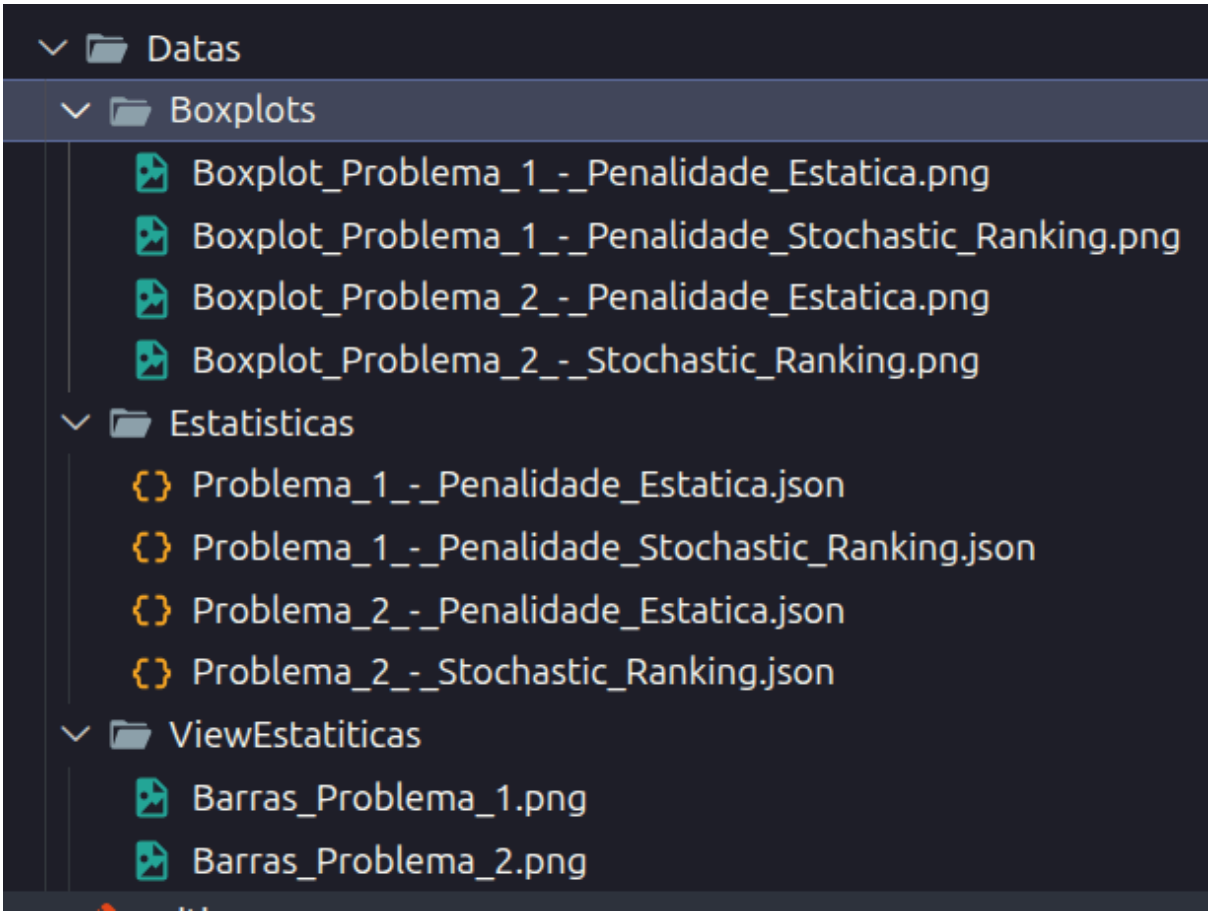
pasta raiz

data/:

Pasta reservada para armazenar resultados e artefatos gerados pelas execuções. Após execução local do script serão gerados os arquivos:

- Estatisticas :Dados salvos em json sobre as estatísticas iterativas de cada problema.
- Boxplots/: Diretório com Gráficos boxplots separados por algoritmo e “tipo do problema”
- ViewEstatisticas: Gráfico das estatísticas iterativas.





Arquivos .py

- **PE_ER_Problem01.py:** Implementação do Problema 1 utilizando Programação Evolutiva - Estática Restrição (ER). Usa penalidades fixas (multas estáticas) para lidar com violações de restrições. Gera estatísticas e resultados em formato JSON.

- **PE_ER_Problem02.py:** Implementação do Problema 2 utilizando Programação Evolutiva - Estática Restrição (ER). Usa penalidades estáticas para garantir a viabilidade das soluções. Salva resultados das execuções em arquivos JSON.

- **PE_SR_Problem01.py:** Implementação do Problema 1 utilizando Stochastic Ranking (SR). Utiliza classificação estocástica para ordenar soluções, oferecendo uma abordagem alternativa ao tratamento de restrições. Gera estatísticas das execuções em json.

- **PE_SR_Problem02.py:** Implementação do Problema 2 utilizando Stochastic Ranking (SR). Emprega o método de classificação estocástica para avaliar soluções viáveis. Armazena resultados em formato JSON.

- **geradorGraficos.py:** Utilitário responsável por gerar visualizações dos resultados das execuções. Carrega dados JSON das estatísticas, cria boxplots para análise de convergência e gera gráficos de barras comparando métricas (mínimo, mediana, máximo e desvio padrão) entre os diferentes algoritmos e problemas.

readme.md

Arquivo de documentação onde pode-se encontrar a visão geral do projeto, instruções de uso e objetivos.

.gitignore

Arquivo de controle para versionamento.

3. Desenvolvimento

Esta seção descreve as principais decisões tomadas durante a implementação das metaheurísticas, com foco em escolhas de inicialização, critérios de parada e configurações específicas de cada algoritmo.

Soluções iniciais:

As soluções iniciais são geradas de forma estocástica (aleatórias). Sendo assim, para cada iteração das 30 soluções por algoritmo haverá soluções quaisquer, pertencentes a um grupo de factíveis e não factíveis.

Critérios de parada:

O encerramento das execuções ocorre por limite de iterações, ou seja quando o número máximo de gerações (filhos) são alcançados

Mutações:

Diferentemente dos Algoritmos Genéticos, a Programação Evolutiva não utiliza os operadores de cruzamento (crossover). Cada pai gera um único descendente (filho) apenas sofrendo uma alteração independente em seus próprios "genes" (variáveis). Nos códigos implementados com Penalidade Estática (*PE_ER_Problem01.py* e *PE_ER_Problem02.py*), a mutação é realizada através da adição de um Ruído Gaussiano com Decaimento Dinâmico, o que significa que o passo da mutação (σ) diminui progressivamente ao longo das gerações.

- Ruído: O algoritmo aplica uma perturbação aleatória baseada na Curva de Sino (Distribuição Normal ou Gaussiana) da biblioteca Numpy, com média zero.
- Passo Inicial (Exploração): No início da execução, o desvio-padrão (σ) começa configurado para abranger 10% do tamanho total do espaço de busca. Isso permite que as partículas deem saltos amplos o suficiente para explorar o mapa sem cair excessivamente nas zonas de multas elevadas geradas pela penalidade estática.
- Decaimento (Explotação): A cada geração, a força da mutação sofre um decaimento linear. Nas últimas gerações, a mutação atua como um "ajuste microscópico" focado apenas no refinamento das casas decimais. Esse

polimento final é essencial para que o algoritmo consiga cravar as restrições rígidas do problema, especialmente as restrições de igualdade (atingindo a tolerância do ε)."

Restrições:

A dificuldade principal na Programação evolutiva reside em garantir que a solução gerada seja factível, ou seja, que ela obedeça a todas as leis físicas, equações de balanço e limites operacionais do projeto.
Nos algoritmos desenvolvidos, as partículas lidam com três barreiras geométricas:

- Restrições de Limites: Fronteiras absolutas das variáveis. São tratadas de forma direta pelo algoritmo de mutação através da função de rebatimento (np.clip), impedindo fisicamente que os indivíduos nasçam fora do espaço de busca global.
- Restrições de Desigualdade (g(x) <= 0): Limites operacionais calculados dinamicamente. Qualquer valor superior a zero é contabilizado como infração.
- Restrições de Igualdade (h(x) = 0): Contornamos isso convertendo a igualdade em uma desigualdade baseada em uma margem de tolerância microscópica (|h(x)| - ε <= 0), onde ε = 0.0001.

Penalidades:

Penalidade Estática (PE_ER): Quando um indivíduo quebra uma regra, o tamanho da sua violação é elevado ao quadrado e multiplicado por um fator de multa muito grande. O resultado é somado ao custo original da função. Com isso, o algoritmo é forçado a buscar a região factível desesperadamente para estancar a multiplicação da multa. Problema: Em problemas onde a região factível é minúscula (como as 11 restrições do Problema 2) ou possui equações de igualdade, os indivíduos entram em convergência prematura, ficando frequentemente presos em sub-ótimos locais apenas para evitar a penalidade extrema.

Stochastic Ranking (PE_SR): A seleção dos sobreviventes ocorre por meio de um torneio guiado por uma probabilidade de factibilidade. Compara dois indivíduos. Se ambos são factíveis, vence o de menor custo. Se ambos são infactíveis, vence o de

menor violação. O trunfo ocorre quando um indivíduo factível enfrenta um infactível: ao invés do indivíduo válido vencer sempre, o algoritmo concede uma chance de ignorar a violação e comparar apenas o custo da função.

- Fase de Exploração (Anarquia): O algoritmo inicia com probabilidade de ignorar restrição alta. Isso permite que soluções com custos excepcionais, mas ligeiramente fora da lei, sobrevivam e cruzem o espaço de busca, contornando montanhas e paredes de restrições.
- Fase de Explotação (Ditadura): Ao longo das gerações, probabilidade de ignorar a restrição decai linearmente. Nas gerações finais, a "vista grossa" para violações é quase extinta, forçando os indivíduos a realizar o ajuste fino nas casas decimais e deslizar para o interior da região estritamente factível.

Seleção e Sobrevivência:

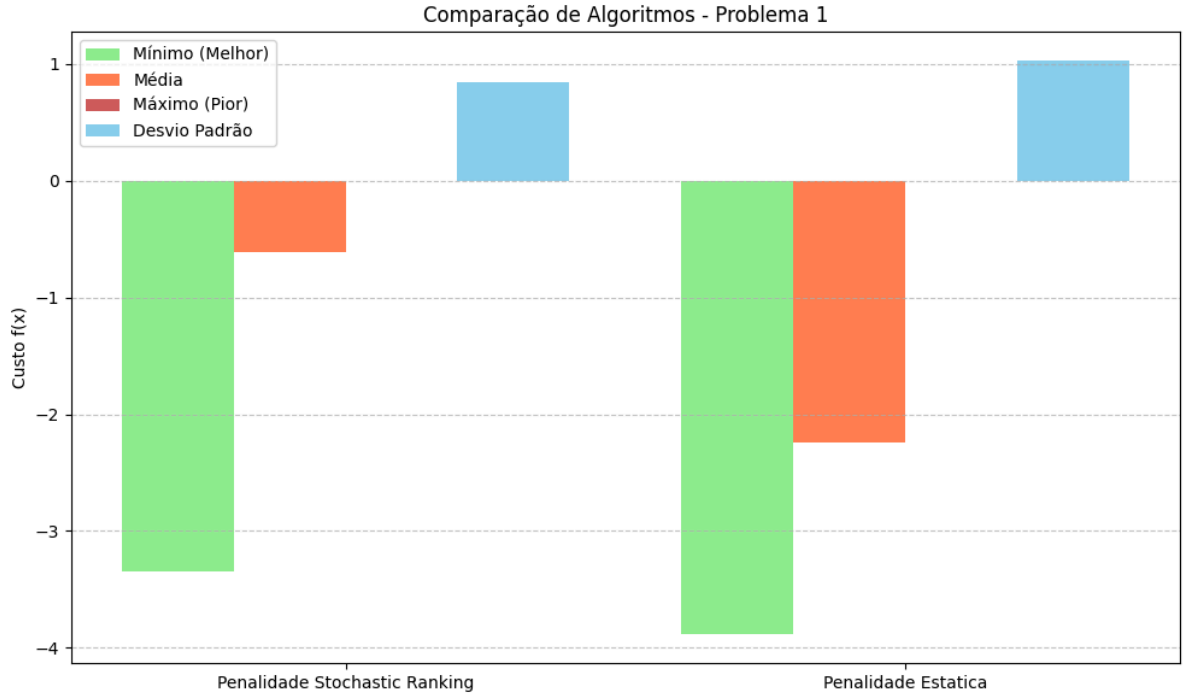
Na Penalidade Estática: Quem tiver o menor valor de Fitness (Custo da função + Multas das restrições) ganha o ponto de vitória. Já no Stochastic Ranking: avalia-se o custo e a violação separadamente. Ganha quem quebrou menos regras ou, se houver empate nas violações, quem tem o menor custo. De vez em quando, existe vista grossa, e é dada a vitória para o indivíduo só ser mais barato.

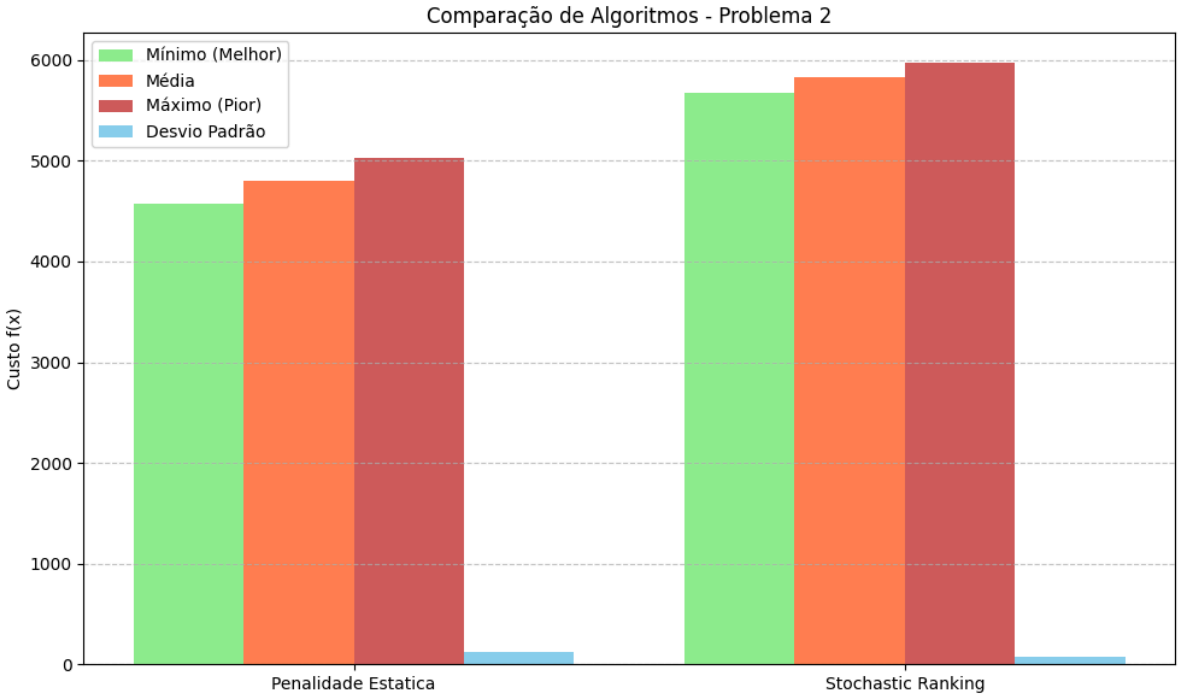
É criado um placar e anoto quantas vitórias cada um dos indivíduos conseguiu. No final, é ordenada a lista do maior vencedor para o menor. O "pódio" só tem espaço para o tamanho original da população. Essa elite sobrevive, tornando-se os "pais" da próxima geração, e a metade perdedora é deletada da memória.

4. Resultados:

Função 1				
Programação Evolutiva	Minimo	Máximo	Média	Desvio-padrão
Rest. Penalidade Estatica	-3,88793589	-0,00008640	-2,24277731	1,02822357
Rest. Stochastic Ranking	-3,34352677	-0,00006892	-0,61388757	0,84161949

Função 2				
Programação Evolutiva	Minimo	Máximo	Média	Desvio-padrão
Rest. Penalidade Estatica	4.572,33505663	5.031,16035938	4.803,13286965	128,17122066
Rest. Stochastic Ranking	5.669,89969091	5.969,86183780	5.828,48828526	74,69255732





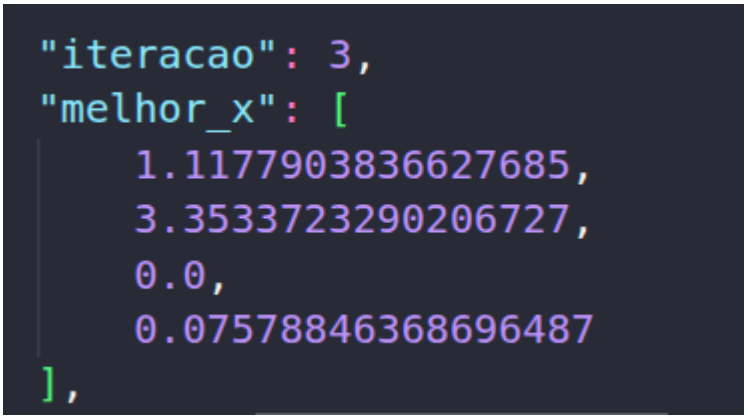
- **Problema 1 - Penalidade Estática:**


```
"iteracao": 17,
"melhor_x": [
    0.895963540678361,
    3.9979900305074687,
    0.4367155530784439,
    0.0005971958850500946
],
```

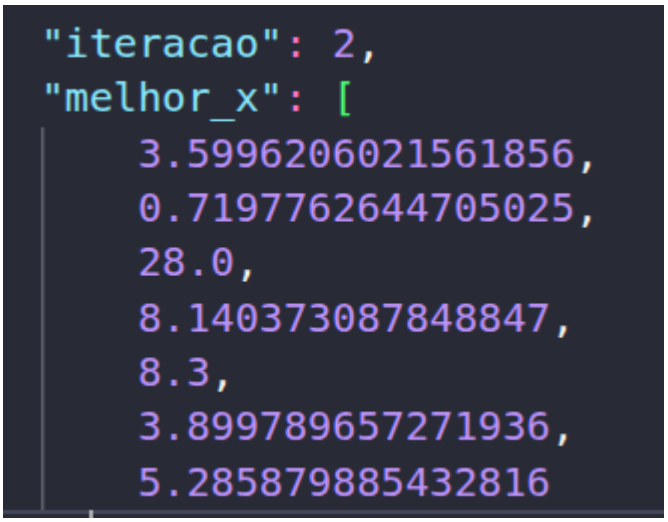

- **Problema 2 - Penalidade Estática:**


```
"iteracao": 11,
"melhor_x": [
    3.513997207385744,
    0.7002075433705305,
    24.45417700981014,
    7.813785581083595,
    8.3,
    3.3893066765307083,
    5.286067488460811
],
```

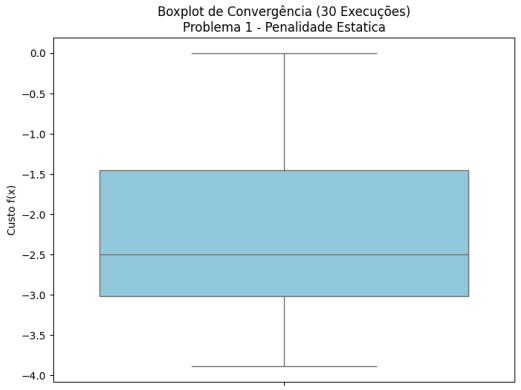
- **Problema 1 - Stochastic Ranking:**

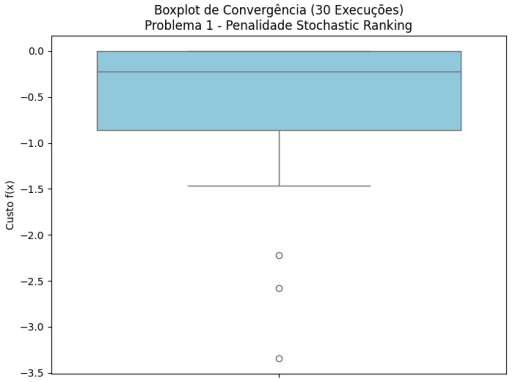


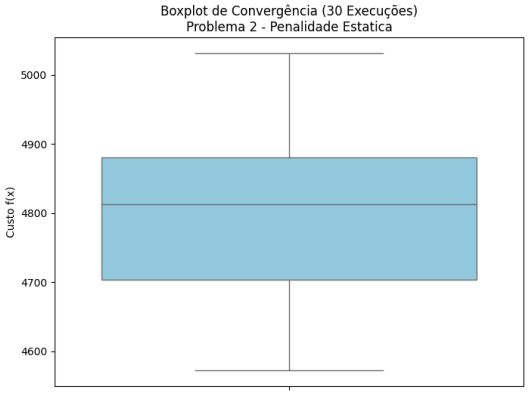
- **Problema 2 - Stochastic Ranking:**

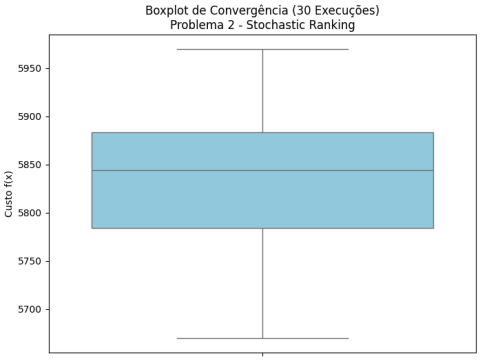


Análise de Boxplots









- **Problema 1 - Penalidade Estática:** O algoritmo apresenta uma alta dispersão de resultados, evidenciada pela caixa (Intervalo Interquartil) bastante larga.A mediana de convergência se estabeleceu em torno de -2.5.O longo "bigode" (whisker) inferior indica que o algoritmo conseguiu explorar regiões de custo excelentes (chegando a quase -4.0), mas a alta variância sugere que ele tem dificuldade em manter a consistência, parando em diferentes ótimos locais a cada execução.
- **Problema 2 - Penalidade Estática:** Os custos variaram aproximadamente entre 4580 e 5030, com a mediana centralizada perto de 4810.A distribuição é visualmente simétrica e consistente (sem outliers). No entanto, em problemas altamente restritos, esse comportamento estável com uma penalidade rígida geralmente indica que a população ficou encurralada em uma mesma fronteira do espaço de busca.

- **Problema 1 - Stochastic Ranking:** A maior parte das execuções concentrou-se muito próxima ao custo 0.0. Isso ilustra perfeitamente o fenômeno da "Armadilha da Factibilidade", onde o vetor de zeros atrai fortemente os indivíduos .O destaque deste gráfico é a presença de outliers (os pontos fora da curva abaixo de -2.0). Eles provam que, nas vezes em que o algoritmo conseguiu romper a barreira inicial do zero através do decaimento, ele mergulhou profundamente no espaço de busca e encontrou soluções altamente otimizadas.
- **Problema 2 - Stochastic Ranking:** Os indivíduos convergiram para uma faixa de valores diferente, com a mediana estabelecida na casa dos 5840 e variação entre 5670 e 5970.Assim como a Penalidade Estática, não apresentou outliers, mostrando grande estabilidade em suas 30 execuções. A transição para valores nominalmente mais altos evidencia que o Ranqueamento Estocástico guiou as partículas para uma região topológica distinta, priorizando a busca rigorosa pelo cumprimento das 11 restrições de engenharia.

5. Conclusão

A análise estatística de 30 execuções revela que a topologia do problema define a eficácia do método de tratamento de restrições. Na Função 1, caracterizada por uma restrição de igualdade rigorosa e um sub-ótimo enganoso em zero, a Penalidade Estática demonstrou superioridade, alcançando a menor média (-2,24) e o menor valor mínimo (-3,88) ao penalizar severamente os desvios. O Stochastic Ranking, apesar de tentar contornar as restrições, foi frequentemente capturado pela armadilha do zero. Em contrapartida, na Função 2, que possui um espaço de busca denso e restrito por 11 inequações não-lineares, o Stochastic Ranking revelou sua maior força: a robustez. Embora a Penalidade Estática tenha alcançado custos matematicamente menores, ela apresentou alta dispersão (Desvio-Padrão de 128,17). O Stochastic Ranking contornou as restrições mecânicas de forma muito mais coesa, garantindo uma convergência estável e segura em todas as rodadas, reduzindo a variância (Desvio-Padrão de 74,69) e se provando o método mais confiável para problemas complexos de engenharia do tipo (testados).

6. Referências

- Slides - Professor Marcus - UFV
- https://www.ime.usp.br/~gold/cursos/2009/mac5758/AndreBuscaTabu.pdf
- https://pymoo.org/algorithms/soo/sres.html